

BehaviorShield

Technical Build Document

Continuous AI-Driven Behavioral Biometric Authentication

Team SOLARIS | DFS/IBA Cyber Security Hackathon 2026

Document Purpose

Complete technical reference for all 4 team members.
Read fully before writing any code.

Demo Deadline

June 30, 2026 — 20 days from today

01 What Are We Building — The Big Picture

BehaviorShield is a system that watches **how** a user types, moves their mouse, and navigates a banking app — and raises an alarm the moment that behavior stops matching the known user. It runs silently in the background of every banking session without adding any friction for legitimate users.

The core insight: Every person has a unique behavioral fingerprint — typing rhythm, mouse movement, scroll speed — that is very hard to fake even if an attacker has your password and OTP. BehaviorShield captures this fingerprint continuously and scores every moment of the session against it.

The Three-Outcome Decision Engine

Risk Score	What Happens	User Experience
0 – 30 (Green)	Silent pass — session continues normally	Zero friction, user notices nothing
30 – 70 (Amber)	Invisible step-up — biometric re-prompt or silent OTP challenge	Minor friction, shown only to suspicious sessions
70 – 100 (Red)	Session frozen — fraud-ops alerted, account hold triggered	Session ends, re-authentication required

A more detailed breakdown

Great question — right now "minor friction" and "step-up challenge" are vague. Let me define exactly what happens at each score band, specifically enough that your frontend member can build it directly from this spec.

Exact Behavior at Each Score Band

Green Zone: Score 0–30 — Silent Pass

What happens: Absolutely nothing visible. Session continues normally.

Behind the scenes:

- Score logged to dashboard
- Baseline update queued (this session contributes to rolling 30-day profile)
- Next score check scheduled in 30 seconds

User sees: Nothing. Zero friction.

Fraud-ops dashboard shows: Green indicator, score number, session active.

Amber Zone: Score 31–70 — Three Sub-Tiers

This is where most of the nuance lives. A score of 32 and a score of 68 are very different situations and should not trigger the same response.

Amber Low: Score 31–45 — Passive Monitoring Intensified

What happens: Nothing visible to the user. Internally:

- Scoring frequency increases from every 30 seconds to every 10 seconds
- Device fingerprint re-checked silently

- If score stays in this band for 3 consecutive checks → escalate to Amber Mid
- If score drops back below 30 → return to Green, reset frequency

User sees: Nothing.

Why this exists: A score of 35 could just be the user typing on an unfamiliar keyboard or being distracted. No reason to challenge yet — just watch more closely.

Amber Mid: Score 46–60 — Silent Re-Authentication

What happens: A subtle, non-alarming prompt appears. Not a scary security warning — just a gentle re-verification that looks like a normal banking UI element.

Exact UI trigger: A small banner appears at the top of the page:

"For your security, please confirm your identity."

[Type your name or PIN to continue] ← single field, not full login

- The user types their name or a 4-digit PIN they set during enrollment
- SDK captures the behavioral signal from this typing
- If the behavioral signal from THIS typing matches their enrolled baseline → score resets to 15, session continues
- If it doesn't match → score jumps to 65, escalates to Amber High
- User gets 2 attempts before escalation

User sees: A soft, non-alarming prompt. Not a full page takeover. Looks like a normal security check, not an alarm.

Why a name/PIN and not OTP? Because OTP doesn't test behavior — anyone with the phone can enter it. Typing their name gives you another behavioral sample to score against. This is the core insight.

Amber High: Score 61–70 — Transactional Restrictions + Behavioral Challenge

What happens: Two things simultaneously:

1. Transaction restrictions kick in:

Action	Restriction Applied
View balance / statements	✓ Allowed — read-only is low risk
Transfer < ₹1,000	✓ Allowed
Transfer ₹1,000 – ₹10,000	⚠ Requires OTP before proceeding
Transfer > ₹10,000	✗ Blocked — "Service temporarily unavailable"
Change password / mobile number	✗ Blocked
Add new payee	✗ Blocked
UPI payment	⚠ Requires OTP + behavioral re-check

2. Full behavioral re-authentication prompt:

A modal (not full page) appears:

"We noticed unusual activity in this session.

Please re-enter your username and password to continue."

- User re-types their full credentials
- SDK captures full keystroke behavioral profile from this typing
- This is compared against their enrolled login baseline — the strongest behavioral signal you have
- Match → score resets to 20, restrictions lifted, session continues normally
- No match → score pushed to 75, enters Red zone

Why show a fake "service unavailable" for high-value transfers instead of explaining? Because telling a fraudster "you're blocked because risk score is high" teaches them to slow down and be more careful. The message should look like a generic system issue, not a security trigger.

Red Zone: Score 71–100 — Three Sub-Tiers

Red Low: Score 71–82 — Session Frozen, Soft Lock

What happens:

- All actions blocked immediately
- Full-page overlay appears (cannot be dismissed):

 Session Suspended

Your session has been temporarily suspended for security reasons.

No transactions have been processed.

To continue, please log in again from a trusted device.

If you need help: 1800-XXX-XXXX

[Log Out]

- Current session token invalidated immediately
- Fraud-ops dashboard gets alert: score, top 3 contributing features, session timeline
- Account itself is NOT locked — user can log in again from scratch
- If re-login from same device scores Green → session resumes normally (was probably a false positive)
- If re-login also scores Amber/Red → escalates to Red Mid

What fraud-ops sees:

ALERT — Session Frozen

User: [ID] | Score: 78 | Session: 14m 32s

Top contributors:

- Typing speed 55% slower than baseline +38 pts
- New device fingerprint (first seen) +22 pts

- Login attempt at 03:14 AM +18 pts

Action: [Mark False Positive] [Escalate] [Lock Account]

Red High: Score 83–95 — Account Soft Lock + Proactive Alert

What happens:

- Session frozen (same overlay as above)
- Account flagged — next login from any device requires additional verification (OTP + video KYC or branch visit depending on bank policy)
- Automated SMS/email sent to registered mobile/email:

"A suspicious session was detected on your Bharat Suraksha

Bank account at [time]. Your session has been ended for

your protection. If this was you, please call 1800-XXX-XXXX.

If not, your account is safe — no transactions were processed."

- Fraud-ops gets high-priority alert with full session replay
- Account not fully locked — just requires extra step on next login

Red Critical: Score 96–100 — Bot / Scripted Attack Detected

What happens:

- Session terminated in < 300ms (before login even completes if detected at login)
- IP address rate-limited (10 minute cooldown for that IP)
- If same IP hits 3 times → 24 hour block
- No user-facing message — just returns HTTP 429 or generic "Something went wrong"
- Fraud-ops gets bot alert with mouse trajectory data (the straight-line giveaway)
- No account notification sent (it's a bot — there's no real user to notify)

Why no message at all? Telling a bot "you've been blocked" gives the attacker feedback. Silent blocking gives you more time before they adapt.

Complete Decision Matrix

Score	Band	User Sees	Transactions	Fraud-ops Alert	Account Status
0–30	Green	Nothing	All allowed	Score logged	Normal
31–45	Amber Low	Nothing	All allowed	Monitoring flag	Normal
46–60	Amber Mid	Soft name/PIN prompt	All allowed pending re-check	Soft flag	Normal
61–70	Amber High	Re-auth modal	Read + small transfers only	Medium alert	Normal
71–82	Red Low	Session frozen overlay	All blocked	High alert	Session invalidated
83–95	Red High	Session frozen overlay	All blocked	High alert + escalation	Soft lock — extra step on next login

Score	Band	User Sees	Transactions	Fraud-ops Alert	Account Status
96–100	Red Critical	Nothing (silent block)	All blocked	Bot alert	IP rate-limited

What Makes This Different From Existing Solutions

Key Differentiator — Sovereign Deployment

Existing behavioral biometric solutions are typically delivered as managed SaaS — the bank integrates an SDK, and session data is processed on the vendor's infrastructure under a data processing agreement. BehaviorShield takes a different architectural philosophy: the entire pipeline — feature extraction, model inference, and decision engine — runs within the bank's own infrastructure. No session data, feature vectors, or behavioral profiles are transmitted outside the bank's perimeter under any configuration. For Indian PSBs operating under RBI Master Directions on IT 2023 and the DPDP Act 2023, this eliminates the need for third-party data processing agreements entirely."

Dimension	BioCatch (Existing)	BehaviorShield (Ours)
Deployment	SaaS / third-party cloud	On-premise or bank's private cloud
Data leaves bank?	Yes — sent to BioCatch servers	No — feature vectors only, processed locally
Explainability	Black box score	Top contributing features shown to fraud-ops
Cost	Per-user licensing fee	Open-source stack, no licensing
RBI compliance	Requires data sharing agreement	Fully local, no data sharing needed

02 Privacy & Data Collection — The No-Content Principle

CRITICAL — Read This Before Writing Any Code

We **NEVER** collect what the user typed. We collect ONLY timing metadata — how long keys were held and the gaps between them. The actual characters, password content, and key identities are never captured, stored, or transmitted. Violating this makes the system a keylogger, which is illegal and defeats the purpose.

What We Collect vs. What We Don't

Data Type	Collect?	Example Value	Why / Why Not
Key hold duration (ms)	☑ YES	87ms	Timing only — no key identity
Flight time between keys (ms)	☑ YES	42ms	Gap timing — no content
Typing speed (chars/sec)	☑ YES	4.3 cps	Count only — no content
Backspace frequency	☑ YES	0.06 per key	Count only — self-correction rate
Mouse velocity & acceleration	☑ YES	340 px/s	Movement data — no content
Touch pressure & area (mobile)	☑ YES	0.72 normalized	Sensor data — no content
Actual characters typed	✗ NEVER	—	Would make it a keylogger
Which keys were pressed	✗ NEVER	—	Key identity not needed
Password string	✗ NEVER	—	Illegal, regulatory violation
Form field values	✗ NEVER	—	Privacy violation

How the SDK Actually Works — The Flow

The SDK captures raw key events, immediately extracts timing numbers from them, and discards the raw events. Only the 28 aggregate statistical values are ever sent to the server.

SDK Data Flow (browser-side)

Step 1: keydown event fires → record timestamp only (no key identity)

Step 2: keyup event fires → compute hold time = $\text{keyup_ts} - \text{keydown_ts}$

Step 3: Compute flight time = $\text{this_keydown_ts} - \text{previous_keyup_ts}$

Step 4: Append timing values to local array

Step 5: After field submission → extract 28 features from timing array

Step 6: Discard raw timing array — only 28 numbers remain

Step 7: Send 28 numbers to Risk API via encrypted call — nothing else

The 28 Features We Extract

These are the exact 28 features that go into the ML model. Every team member should know this list.

Keystroke Features (15)

#	Feature Name	What It Measures	Notes
1	mean_hold_time	Average key press duration (ms)	Core rhythm signal
2	std_hold_time	Consistency of press duration	Low std = very consistent typist
3	mean_flight_time	Average gap between keys (ms)	Speed of transitions
4	std_flight_time	Consistency of key gaps	Irregular = possible attacker
5	typing_speed_cps	Characters typed per second	Overall pace
6	backspace_rate	Backspaces per total keystrokes	Self-correction tendency
7	rhythm_cv	Coefficient of variation of hold times	Rhythm consistency score
8	burst_ratio	Ratio of fast clusters to slow pauses	Burst typing pattern
9	hold_first_half	Mean hold time in first 50% of input	Opening rhythm
10	hold_second_half	Mean hold time in second 50% of input	Closing rhythm
11-15	pos_digraph_1 to 5	Flight time between positions N and N+1 for positions 1-5	Positional rhythm — no key identity needed

Mouse / Touch Features (8)

#	Feature Name	What It Measures	Notes
16	mouse_mean_velocity	Average cursor speed (px/s)	Natural vs hurried movement
17	mouse_std_velocity	Speed consistency	Bots are unnaturally smooth
18	mouse_mean_accel	Average acceleration	Human movement has natural accel curves
19	click_frequency	Clicks per minute	Interaction pace
20	click_interval_cv	Consistency of click intervals	Human vs bot click pattern
21	idle_ratio	% of session with no mouse movement	Reading/thinking time
22	scroll_speed_mean	Average scroll velocity	Navigation style
23	trajectory_straightness	Actual path / straight-line distance ratio	Curved=human, straight=bot

Session-Level Features (5)

#	Feature Name	What It Measures	Notes
24	time_of_day_norm	Hour of login normalized 0–1	3AM login = anomaly
25	device_fp_score	Known device = 1.0, new device = 0.0	Hash-based, no PII
26	session_action_rate	Pages/actions per minute	Rushed = suspicious
27	txn_initiation_delay	Seconds from login to first transaction attempt	Normal users browse first
Feature	Laptop Demo	Phone Demo	Recommendation
Touch Pressure	✗ Never	✗ Almost never	Drop from active features
Touch Contact Area	✗ Never	☑ Works	Auto-activate only on touch devices
Click Dwell Time	☑ Always	☑ Works	Primary Feature 28 for demo

The last 28 features it should be automatically choice based on the device

03 System Architecture — How All 10 Files Connect

BehaviorShield is built across 10 files split across 4 layers. Every file has one clear responsibility. Here is the full map.

File	Layer	Owner	Responsibility
constants.py	Config	ML Member	All 28 feature names, MIN_STD_FLOOR values, thresholds, model params
ml_engine.py	ML	ML Member	Z-Score profiler, Isolation Forest, risk score fusion, explainability
database.py	Data	Backend Member	In-memory store for user profiles, sessions, event log
app.py	Backend	Backend Member	FastAPI — 8 REST endpoints + 1 WebSocket stream
sdk.js	Client	Dashboard Member	Browser-side signal capture, feature extraction, API calls
style.css	Design	Frontend Member	Shared dark-mode design system for all HTML pages
index.html	Launcher	Frontend Member	Project intro page — entry points for demo
bank.html + bank.js	Banking App	Frontend Member	Bharat Suraksha Bank — enrollment, login, transfer, step-up, freeze
dashboard.html + dashboard.js	Operations	Dashboard Member	Live fraud-ops dashboard — risk timeline, feature breakdown, alerts

Data Flow — End to End

Complete Request Flow (Login Session)

- User opens Bharat Suraksha Bank** (bank.html) → sdk.js loads silently
- User types login credentials** → sdk.js captures hold/flight times only (no key identity)
- sdk.js extracts 28 features** → discards raw events → sends feature vector to POST /session/score
- app.py receives vector** → calls ml_engine.score_session(user_id, features)
- ml_engine.py computes Z-Score** per feature vs. user baseline → Isolation Forest on mouse features → fuses scores → returns Risk Score 0–100 + top 4 contributing features
- app.py decision engine** → score < 30: return {action: 'pass'} | 30–70: return {action: 'stepup'} | >70: return {action: 'freeze', reason: [...]}
- bank.js receives response** → renders appropriate UI (nothing / step-up modal / freeze screen)
- WebSocket pushes score** to dashboard.js in real time → chart updates, alert feed updates

The ML Pipeline — Z-Score Profiler Explained Simply

The Z-Score profiler answers one question: how far is this behavior from what we expect from this specific user?

Z-Score Formula

$Z = (\text{observed_value} - \text{user_mean}) / \text{user_std}$

Example: User's enrolled mean hold time = 100ms, std = 10ms
Legitimate login: hold time = 105ms → $Z = (105-100)/10 = 0.5 \rightarrow$ normal
Attacker login: hold time = 145ms → $Z = (145-100)/10 = 4.5 \rightarrow$ strong anomaly

The system computes a Z-Score for each of the 28 features. The top contributors (highest absolute Z-scores) are surfaced to the fraud-ops dashboard as the explanation for the risk score.

CRITICAL — MIN_STD_FLOOR (prevents division by zero)

During enrollment, early sessions may produce a standard deviation of 0 (not enough data). Dividing by 0 crashes the system. Every feature must have a minimum std floor defined in constants.py:

mean_hold_time: MIN_STD = 8ms
mean_flight_time: MIN_STD = 5ms
typing_speed_cps: MIN_STD = 0.3
mouse_mean_velocity: MIN_STD = 20 px/s
All others: MIN_STD = 0.05 (normalized features)

In code: **`std = max(calculated_std, MIN_STD_FLOOR[feature_name])`**

Isolation Forest — For Mouse and Bot Detection

The Isolation Forest handles mouse dynamics and bot detection. It works differently from Z-Score — instead of comparing to a user baseline, it learns what legitimate human mouse movement looks like in general, and flags behavior that is unusually easy to isolate (i.e., outlier).

Training: Pre-train on BALABIT Mouse Dynamics Dataset (10 users, ~1M events, labeled legitimate vs. bot). This gives us a model that knows what human mouse movement looks like before any user enrolls.

Inference: At runtime, mouse features (features 16–23) are passed to the Isolation Forest. It returns an anomaly score 0–1. Score > 0.7 = strong bot indicator.

Why this works for bots: Bots move in perfectly straight lines with constant velocity and 0ms reaction time. These are extreme outliers in the Isolation Forest's learned space of human behavior.

Score Fusion — How Z-Score + Isolation Forest Become One Number

Component	Weight	What It Contributes
Keystroke Z-Score (normalized)	45%	Per-user typing rhythm comparison
Mouse Isolation Forest score	25%	Human vs. bot mouse detection

Component	Weight	What It Contributes
Session metadata score	20%	Time-of-day, device fingerprint, action rate
Touch pressure Z-Score (mobile)	10%	Mobile-specific behavioral signal

Final Risk Score = weighted sum of components, scaled to 0–100. Each component also contributes its top feature to the explanation output.

04 File-by-File Specification — What Each Member Builds

constants.py — ML Member — Day 1 (Write This First)

This is the first file to write. Nothing else can be built until these values are defined. It contains all thresholds, feature names, and MIN_STD_FLOOR values.

Contents of constants.py

FEATURE_NAMES — ordered list of all 28 feature names (must match sdk.js extraction order exactly)

MIN_STD_FLOOR — dict of minimum std per feature (prevents division by zero)

RISK_THRESHOLDS — {pass: 30, stepup: 70} — the decision boundaries

WEIGHTS — {keystroke: 0.45, mouse: 0.25, session: 0.20, touch: 0.10}

ISOLATION_CONTAMINATION — 0.1 (10% of training data expected to be anomalous)

ENROLLMENT_MIN_SESSIONS — 3 (minimum sessions before behavioral scoring is active)

BASELINE_WINDOW_DAYS — 30 (rolling window for baseline recalibration)

DEMO_PASSPHRASE — the fixed passphrase used for demo enrollment (decide this on Day 1)

ml_engine.py — ML Member — Days 2–4

The core ML module. All scoring logic lives here. app.py calls this module — ml_engine.py never knows about HTTP requests.

Function	Inputs	Returns	Notes
enroll_session(user_id, features)	user_id: str, features: dict[28]	Updated profile dict	Adds session to user's baseline. After ENROLLMENT_MIN_SESSIONS, baseline is active.
score_session(user_id, features)	user_id: str, features: dict[28]	{score: int, action: str, explanation: list}	Main scoring function. Returns 0–100 + decision + top 4 contributors.
compute_keystroke_zscore(profile, features)	profile: dict, features: dict	float 0–1	Z-Score for keystroke features 1–15. Uses MIN_STD_FLOOR.
compute_mouse_score(features)	features: dict	float 0–1	Isolation Forest inference on features 16–23.
compute_session_score(features)	features: dict	float 0–1	Rule-based scoring for features 24–27.
explain_score(feature_zscores)	dict of {feature: zscore}	list of top 4 contributors	Sorts by abs(zscore), returns top 4 with deviation and direction.
train_isolation_forest()	None	Fitted IsolationForest model	Run once at startup. Uses BALABIT dataset CSV.
update_baseline(user_id, features)	user_id: str, features: dict	None	Rolling 30-day baseline update after each legitimate session.

database.py — Backend Member — Day 2

In-memory data store. No SQL, no files. A Python module with global dicts and simple CRUD functions. Resets on server restart — that is fine for demo.

Data Structure	Schema	Purpose
USER_PROFILES	{user_id: {mean: dict, std: dict, session_count: int, enrolled: bool}}	Behavioral baselines per user
ACTIVE_SESSIONS	{session_id: {user_id, start_time, current_score, status, events: []}}	Live session state
ALERT_LOG	[(ts, user_id, session_id, score, action, explanation, resolved: bool)]	Fraud-ops alert feed
DEVICE_REGISTRY	{user_id: [fingerprint_hash, ...]}	Known devices per user

app.py — Backend Member — Days 2–3

FastAPI server. 8 REST endpoints and 1 WebSocket. Handles HTTP only — all ML logic is in ml_engine.py.

Endpoint	Method	Payload	Returns	Used By
POST /enroll	POST	{user_id, features[28]}	{enrolled: bool, session_count: int}	bank.js during enrollment flow
POST /session/start	POST	{user_id, device_fp}	{session_id, status}	bank.js on login
POST /session/score	POST	{session_id, features[28]}	{score, action, explanation}	bank.js continuously during session
POST /session/end	POST	{session_id}	{summary}	bank.js on logout
GET /profile/{user_id}	GET	—	{baseline stats, enrolled, session_count}	dashboard.js — show enrollment info
GET /alerts	GET	—	[alert objects]	dashboard.js — alert feed
POST /alerts/{id}/resolve	POST	—	{resolved: bool}	dashboard.js — analyst clears alert
GET /demo/reset	GET	—	{ok}	Demo mode — resets all data for clean demo
WS /ws/live	WebSocket	—	Streams {session_id, score, action, ts} events	dashboard.js — live chart

sdk.js — Dashboard Member — Days 5–6

Runs silently in the browser. Captures behavioral signals and sends feature vectors to the backend. The user never sees this running.

SDK Responsibilities

Keystroke capture: Listen for keydown/keyup on all form fields. Store timestamps only. Never store e.key, e.code, or e.keyCode.

Mouse capture: Track mousemove (velocity, trajectory), click events (frequency, intervals), scroll events (speed).

Feature extraction: After each form field submit, extract all 28 features from the collected timing data. Discard raw events immediately.

Session management: Call POST /session/start on login. Call POST /session/score every 30 seconds during session and on every page navigation. Call POST /session/end on logout.

Response handling: On action='stepup': trigger step-up modal in bank.js. On action='freeze': trigger session freeze screen.

bank.html + bank.js — Frontend Member — Days 7–10

Bank name: Bharat Suraksha Bank — sounds like a real PSB, no trademark issues.

Screen / State	What It Shows	Key Interactions
Enrollment screen	5-session enrollment flow with progress indicator	User types passphrase 5 times — sdk.js collects baseline
Login screen	Standard bank login (username + password fields)	sdk.js capturing from the moment focus enters the fields
Dashboard / home	Account balance, quick transfer, recent transactions	sdk.js scoring every 30s via POST /session/score
Step-up modal	Overlay: 'Unusual activity detected. Please verify identity.'	Triggered by sdk.js receiving action='stepup'. User types passphrase again.
Session freeze screen	Full-screen alert: 'Session suspended for security review.'	Triggered by action='freeze'. Shows contact number, log-out button only.
Transfer flow	Simple form: recipient, amount, confirm	High-value transfers trigger an extra score check before confirm

dashboard.html + dashboard.js — Dashboard Member — Days 11–13

The fraud-ops dashboard. This is what judges stare at during the demo. It must be visually impressive and update in real time via WebSocket.

Panel	What It Shows	Implementation
Live Risk Timeline	Line chart — risk score over session duration (the heartbeat graph)	Chart.js line chart, updated via WebSocket every event
Current Score Gauge	Large circular gauge showing current risk score with color coding	CSS/SVG gauge, green/amber/red
Feature Breakdown	Bar chart of top 4 contributing features and their Z-scores	Chart.js bar chart, updated per score event

Panel	What It Shows	Implementation
Enrolled User Profile	Table showing user's baseline ranges for each feature	GET /profile/{user_id} on session start
Alert Feed	Scrolling list of triggered alerts with score, reason, timestamp	Appended on every freeze/stepup event via WebSocket
Session Info	Current user, session duration, device fingerprint match status	Static per session, shown in header
Resolve Button	Analyst can mark an alert as resolved	POST /alerts/{id}/resolve

05 20-Day Build Timeline — Day by Day

Days	Phase	Who	Deliverable	Done When
Day 1	Foundation	All	constants.py written. 28 features formally defined. Passphrase chosen. Datasets downloaded. Roles confirmed.	constants.py committed, team reviewed
Day 2	Backend skeleton	Backend	database.py complete. app.py skeleton with all 8 endpoints stubbed (return mock data for now).	Server starts, endpoints return 200
Day 3	Dashboard shell	Dashboard	dashboard.html shell with hardcoded fake data. WebSocket connection stubbed. Risk chart renders.	Chart shows fake scores updating
Day 4–5	ML engine	ML	ml_engine.py complete. Z-Score profiler working. Isolation Forest trained on BALABIT data. unit tests passing.	score_session() returns real scores
Day 6–7	SDK	Dashboard	sdk.js complete. Captures keystrokes (timing only), mouse, session events. Feature extraction tested.	28 features extracted from a test session
Day 8–10	Banking portal	Frontend	bank.html/js complete. All 5 screens working. sdk.js integrated. API calls wired.	Full login → browse → transfer flow works
Day 11	Wire ML to backend	Backend + ML	app.py calls real ml_engine. POST /session/score returns real risk scores.	End-to-end flow: browser → score → response
Day 12	Decision engine	Backend	Step-up and freeze actions triggered correctly. bank.js renders step-up modal and freeze screen.	Attacker takeover demo triggers freeze
Day 13	Dashboard live data	Dashboard	WebSocket live. Risk timeline updates in real time. Feature breakdown and alert feed working.	Dashboard updates during live session
Day 14–15	Integration testing	All	Full demo loop tested end-to-end. All 4 personas working. Bugs fixed.	Demo runs without errors 3x in a row
Day 15–16	Pre-bake personas	All	3 demo personas enrolled with stable baselines. Demo mode toggle added. Backup JSON sessions saved.	Each persona gives consistent scores
Day 17	Explainability	ML + Dashboard	Feature breakdown panel shows real Z-score contributors. Explanation appears on freeze screen.	Freeze shows 'reason: slow typing + new device'
Day 18–19	Presentation	All	Phase 2 slide deck. Architecture diagram. Practice demo 3x full run-through.	Demo fits in 8 minutes, no crashes
Day 20	Buffer + video	All	Record backup demo video (3 min). Fix any last issues. Travel prep.	Backup video exported and saved offline

06 Demo Script — The Exact 8-Minute Flow

Pre-Demo Setup (before jury enters room)

1. Server running locally (no internet dependency)
2. Persona A (legitimate user) already enrolled with stable baseline
3. Dashboard open on second screen or second tab
4. Backup demo video saved offline on USB
5. Two team members positioned: one at banking portal, one at dashboard

Minute 0–1: Context Setting

Speaker says: 'Banks verify identity once at login. After that, any stolen session is fully trusted for its entire duration. BehaviorShield changes that — it continuously scores every moment of the session and acts in real time, invisibly, with zero friction for legitimate users.'

Show: Landing page (index.html) — architecture overview slide or diagram

Minute 1–2: Enrollment

Speaker: 'First, the system learns what the legitimate user looks like. This takes 3 sessions.'

Show: Enrollment screen — Persona A types passphrase 3 times. Dashboard shows baseline building. Risk score at 0 (no baseline yet).

Point out: 'We capture only timing — hold duration and key gaps. The actual characters are never recorded.'

Minute 2–4: Legitimate Session (Green)

Persona A logs in, browses, checks balance, starts a transfer.

Dashboard: Risk score stays 8–22 throughout. Feature breakdown shows all Z-scores near 0. No alerts.

Speaker: 'Persona A's session — risk score stable at 15. Zero friction, no OTP, no challenge. The system is completely invisible.'

Minute 4–6: Mid-Session Takeover (The Money Shot)

Persona A is mid-session. Persona B (different team member) takes the keyboard.

Dashboard: Score climbs: 15 → 28 → 41 (step-up triggers) → 55 → 79 (session freeze). Alert appears in feed.

Banking portal: Step-up modal fires at 41. Persona B fails the behavioral re-challenge. Score hits 79. Freeze screen appears.

Dashboard: Alert shows: 'Session frozen — top contributors: typing speed 40% slower (+34pts), new device fingerprint (+18pts), unusual rhythm (+16pts)'

Speaker: 'The attacker had the correct password and was already logged in. BehaviorShield detected the takeover in 23 seconds purely from behavioral signals. Here is exactly why it fired.'

Minute 6–7: Bot Attack

Run the bot script against the login endpoint.

Dashboard: Score hits 95 in under 2 seconds. Isolation Forest flag. Session blocked before login completes.

Speaker: 'Bots move in straight lines with constant velocity and 0ms reaction time. The Isolation Forest trained on real human mouse data flags this instantly.'

Minute 7–8: Q&A Prep

Hand back to jury. Have answers ready for: cold start, behavioral drift, false positive rate (cite your real test numbers), scalability, regulatory compliance, difference from BioCatch.

07 Jury Q&A — Prepared Answers

Question	Answer
What about the first session? (Cold start)	We pre-train on the CMU Keystroke Dynamics Benchmark Dataset which gives population-level priors. For new users, we start with a conservative threshold weighted toward device fingerprint and IP risk layers. Behavioral scoring activates after 3 sessions as the personal baseline builds.
What if the user changes their behavior? (Injury, stress, new device)	Gradual drift is absorbed by a rolling 30-day baseline recalibration — the model continuously updates. A sudden drastic change triggers step-up authentication but does not lock the account. Re-authentication resets the baseline window.
How is this different from BioCatch?	BioCatch is a black-box SaaS — behavioral data leaves the bank's infrastructure to a third-party server. BehaviorShield runs entirely within the bank's own perimeter, complying with RBI Master Directions on IT 2023. Additionally our explainability layer surfaces the exact features that triggered each decision — BioCatch is opaque to fraud analysts.
What is your false positive rate?	In our internal testing across [X] sessions with [Y] personas, we measured a false positive rate of [Z]% — fill with real numbers from testing in Days 14–16. The Z-Score threshold is configurable — banks can tune sensitivity vs. friction based on their risk appetite.
Why not just use OTP forever?	OTP adds 15–30 seconds per transaction. At 21 billion UPI transactions per month, even 1% OTP friction costs 210 million frustrated user moments. OTP also protects only the login — a stolen session cookie bypasses it entirely. SIM-swap fraud specifically targets OTP delivery. BehaviorShield is invisible to legitimate users — zero added time.
Can it scale to millions of users?	The inference microservice is stateless — it takes a 28-number feature vector in and returns a score. It scales horizontally on Kubernetes. The per-user profile stored is just 56 floating-point numbers (28 means + 28 stds) — negligible storage at any scale.
What about users with motor disabilities?	Users with disabilities that affect their motor patterns can opt into a fallback MFA path — standard OTP or hardware token — without any security degradation. The system flags them as requiring the alternative path during enrollment.
What data do you store about users?	We store 28 mean values and 28 standard deviation values per user — purely statistical aggregates of timing metadata. No keystrokes, no characters, no biometric templates in the traditional sense. This is compliant with DPDP Act 2023 and RBI IT Master Directions.

08 Future Enhancements — Production Roadmap

These are features deliberately scoped out of the prototype to ensure demo reliability. Every item listed here replaces or upgrades something in the current implementation.

Enhancement	What It Replaces	Why Deferred	Target Timeline
LSTM Autoencoder (PyTorch)	Z-Score profiler	Needs much more training data and convergence time than 20 days allow	6–12 months post-hackathon
XGBoost + SHAP fusion	Weighted average score fusion	Requires labeled fraud data for training; more complex to debug under demo pressure	3–6 months
Federated Learning (Flower)	Local model updates only	Requires multi-node infrastructure — too complex for prototype	12–24 months
Apache Kafka + Spark pipeline	WebSocket + in-memory	Production-scale event streaming requires DevOps infrastructure beyond prototype scope	Pre-production phase
Native Kotlin + Swift SDK	Browser JS SDK only	Mobile SDK requires device lab testing and platform-specific tuning	6–9 months
PostgreSQL + TimescaleDB	In-memory Python dicts	Persistence and time-series queries needed at production scale	Pre-production phase
UPI platform integration	Internet Banking demo only	UPI's API requires NPCI partnership and sandbox access — not available in 20 days	18–30 months
Adversarial robustness testing	—	Replay attack defense and behavioral spoofing require red-team testing	Ongoing post-launch